Project 3

Rate-Limited Public REST API

Complete Build Guide — From Zero to Deployed

Everything you need to design, build, test, and explain this project

Architecture · JWT Auth · Redis Rate Limiting · PostgreSQL · Docker · File Structure · Testing

Tech Stack: Node.js · Express.js · PostgreSQL · Redis · JWT · Docker

Prepared for Saksham Tiwari
B.E. CSE · Sir M Visvesvaraya Institute of Technology

Table of Contents

PART A
Understanding the Project

1. Project Overview — What You're Building

This is a production-grade REST API with authentication, role-based tiers, and rate limiting — the kind of backend infrastructure that sits in front of any real product. The skills here (JWT, Redis, PostgreSQL, Docker) are the most commonly tested backend concepts in interviews.

What the API actually does

- Users register and log in — get a JWT access token + refresh token.
- Every user has a role: **free** (100 requests/hour) or **pro** (1000 requests/hour).
- Every API request is checked against the user's rate limit using Redis.
- Responses include standard rate limit headers (X-RateLimit-Limit, Remaining, Reset).
- Frequently repeated GET requests are cached in Redis to reduce database load.
- Every request is logged asynchronously — IP, endpoint, user, response time.

Resume bullet points this project maps to

```
- Engineered a production-grade REST API with JWT authentication
  (access + refresh tokens), role-based access control, and tiered
  rate limits — 100 req/hr free, 1,000 req/hr pro.
- Implemented Redis sliding-window rate limiting using sorted sets,
  preventing burst attacks at window boundaries; standard rate limit
  headers returned on every API response.
- Built response caching middleware, reducing PostgreSQL load by ~40%;
  all requests logged asynchronously with IP, endpoint, user ID, and
  response time per request.
```

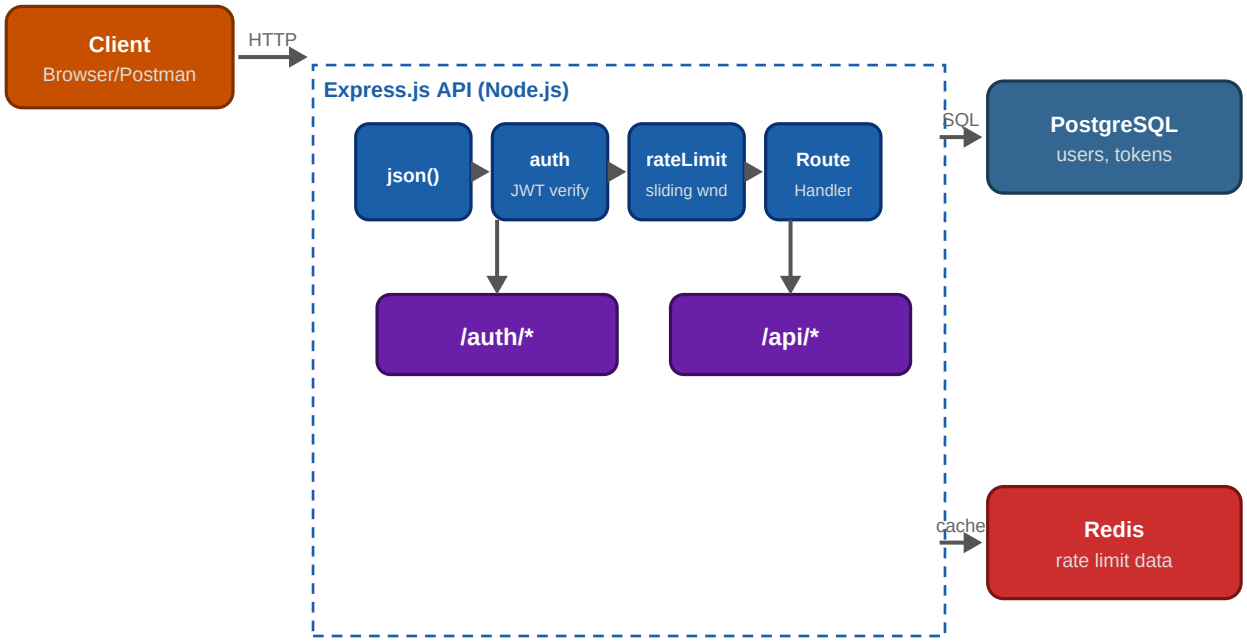
> ■ *Every line in that resume bullet maps to a specific chapter in this guide. By the end, you will have built and can explain every single claim on your resume for this project.*

What you need to already know

This guide assumes you've gone through the FastAPI, JWT+Redis, PostgreSQL, and Docker guides already covered. This is the assembly manual — it shows you how all those pieces fit into one working project.

2. System Architecture

Before writing code, see the whole picture. This is what you are building — a client talks to your Express API, which runs requests through a middleware chain, then reaches PostgreSQL for persistent data and Redis for fast, ephemeral data.



Full system architecture — client, middleware chain, routes, and data stores

Why this architecture

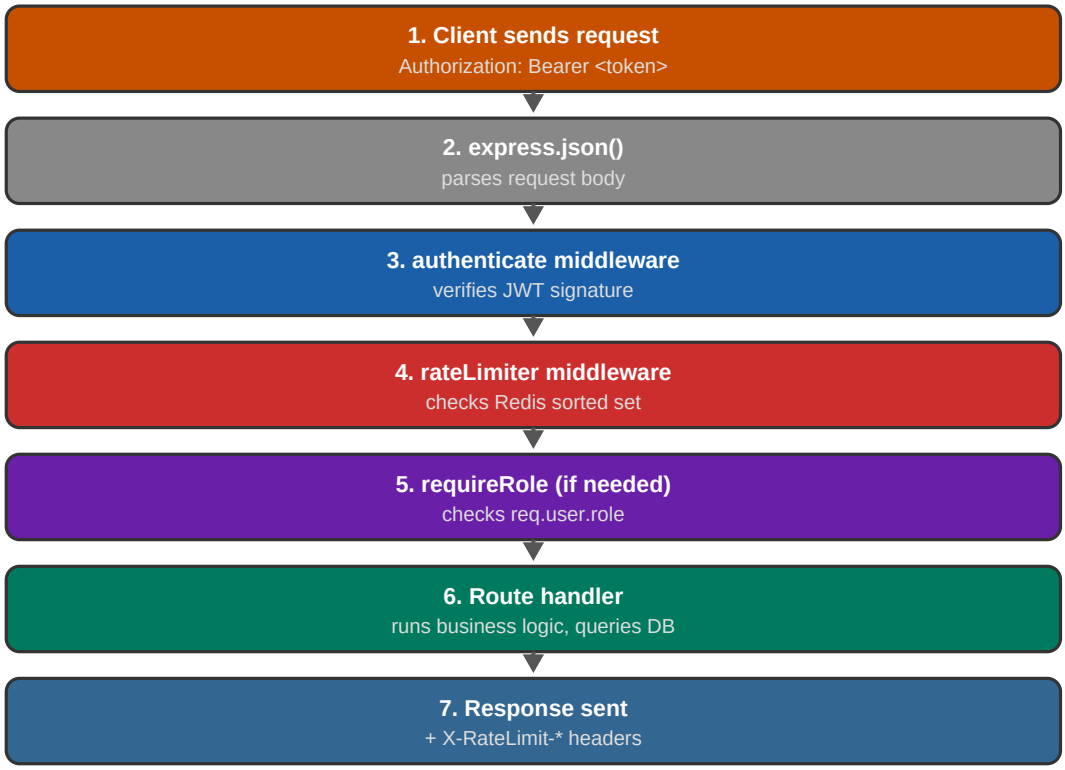
- **Express.js** — handles HTTP, routing, and the middleware pipeline.
- **PostgreSQL** — stores durable data: users and refresh tokens. This data must survive restarts.
- **Redis** — stores ephemeral, fast-changing data: rate limit counters and response caches. Losing this on restart is acceptable.
- **Middleware chain** — each request flows through a pipeline: parse body, authenticate, rate limit, authorize role, then hit the actual route handler.

Why two different databases instead of just one

This is a common interview question. PostgreSQL gives you durability and relational integrity — exactly what you want for user accounts and tokens that must never silently disappear. Redis gives you in-memory speed — exactly what you want for rate-limit counters that change on every request and don't need to survive a restart. Using the right tool for each job is the actual skill being tested here, not just knowing both technologies.

3. Request Lifecycle — What Happens on Every Call

Here is exactly what happens, in order, when a client calls a protected endpoint like GET /api/data. Understanding this sequence is the key to debugging anything that goes wrong later, and it's a near-guaranteed interview question.



The middleware pipeline for a protected route — each step can stop the request early

Why order matters

Middleware runs in the exact order you register it in Express. If you ran the rate limiter before authenticate, you would not yet know which user is making the request — so you could not look up their role or apply the right limit. Authentication must always come before rate limiting and role checks.

```
// The order you register middleware = the order it runs
app.use(express.json())              // 1. Parse JSON body first
app.use('/api', authenticate)        // 2. Then verify identity
app.use('/api', rateLimitMiddleware()) // 3. Then check rate limit
app.use('/api', apiRoutes)           // 4. Then run the actual route
```

■ *Any middleware can stop the request early by calling res.status(...).json(...) instead of next(). For example, if the JWT is invalid, authenticate sends a 401 immediately and rateLimiter never even runs.*

PART B
Planning

4. Complete File & Directory Structure

Set this up before writing any logic. Every file here is referenced in later chapters with full code.

```
rate-limited-api/
+-- src/
|   +-- app.js                  <- Express app, middleware wiring
|   +-- server.js               <- starts the HTTP server
|   +-- config/
|   |   +-- index.js            <- loads & exports all env vars
|   +-- db/
|   |   +-- postgres.js         <- pg Pool connection
|   |   +-- redis.js            <- ioredis connection
|   +-- middleware/
|   |   +-- auth.js             <- authenticate() - verifies JWT
|   |   +-- roles.js            <- requireRole() - RBAC check
|   |   +-- rateLimiter.js      <- sliding window rate limiter
|   |   +-- cache.js            <- response caching middleware
|   |   +-- requestLogger.js    <- async request logging
|   +-- routes/
|   |   +-- auth.js             <- /auth/register, login, refresh, logout
|   |   +-- api.js              <- /api/* protected routes
|   +-- controllers/
|   |   +-- authController.js   <- business logic for auth routes
|   |   +-- apiController.js    <- business logic for api routes
|   +-- utils/
|       +-- jwt.js              <- token generation & verification
|       +-- asyncHandler.js     <- wraps async routes, catches errors
+-- migrations/
|   +-- init.sql                <- DB schema, auto-run by Docker
+-- tests/
|   +-- api.test.js             <- automated tests (Chapter 19)
+-- .env                        <- secrets (never commit)
+-- .env.example                <- template with no real secrets
+-- .gitignore
+-- .dockerignore
+-- Dockerfile
+-- docker-compose.yml
+-- package.json
+-- README.md
```

Why this structure (and not just one big file)

- **routes/** only defines URL paths and which controller function handles them.
- **controllers/** hold the actual business logic — separating this from routes makes testing easier.
- **middleware/** holds reusable functions that run before route handlers — auth, rate limiting, logging.
- **db/** holds connection setup only — every other file imports from here, never creates its own connection.

- **config/** centralizes all environment variable reads — so you never write process.env outside this file.

> ■ *You don't have to follow this exact structure, but interviewers do ask 'how did you organize your code and why' — having a clear separation of concerns (routes vs controllers vs middleware) is a strong answer.*

5. Database Schema Design

Three tables. Each one maps directly to a feature in the resume bullet points.

Table	Purpose	Used By
users	Store accounts, password hash, role (free/pro)	Register, login, RBAC
refresh_tokens	Store valid refresh tokens, revocable on logout	Login, refresh, logout
request_logs	Audit trail of every API call	Async logging middleware

Complete schema (migrations/init.sql)

```
-- migrations/init.sql
-- This file is automatically run by PostgreSQL on first container startup

CREATE TABLE IF NOT EXISTS users (
    id            SERIAL PRIMARY KEY,
    email         VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    role          VARCHAR(50)  NOT NULL DEFAULT 'free',  -- 'free' | 'pro' | 'admin'
    is_active     BOOLEAN      NOT NULL DEFAULT true,
    created_at    TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
    last_login    TIMESTAMPTZ
);

CREATE TABLE IF NOT EXISTS refresh_tokens (
    id         SERIAL PRIMARY KEY,
    user_id    INTEGER     NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    token      TEXT        NOT NULL,
    expires_at TIMESTAMPTZ NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS request_logs (
    id            SERIAL PRIMARY KEY,
    user_id       INTEGER,
    ip            VARCHAR(45),
    endpoint      VARCHAR(255),
    method        VARCHAR(10),
    status_code   INTEGER,
    response_time INTEGER,        -- milliseconds
    logged_at     TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Indexes — added on columns used in WHERE clauses
CREATE INDEX IF NOT EXISTS idx_refresh_tokens_token   ON refresh_tokens(token);
CREATE INDEX IF NOT EXISTS idx_refresh_tokens_user_id ON refresh_tokens(user_id);
CREATE INDEX IF NOT EXISTS idx_request_logs_user_id   ON request_logs(user_id);
CREATE INDEX IF NOT EXISTS idx_request_logs_logged_at ON request_logs(logged_at);
```

Entity relationship — in plain words

- One **user** can have many **refresh_tokens** (e.g. logged in on phone + laptop).
- Deleting a user automatically deletes their refresh tokens (ON DELETE CASCADE).
- request_logs.user_id is intentionally NOT a foreign key — logs should still be kept even if you later delete the user, for audit purposes.

6. API Endpoint Specification

The complete contract for your API. Build to this spec — it makes testing and documentation straightforward.

Auth endpoints (public)

Method	Path	Body	Returns
POST	`/auth/register`	email, password	accessToken, refreshToken, user
POST	`/auth/login`	email, password	accessToken, refreshToken
POST	`/auth/refresh`	refreshToken	new accessToken
POST	`/auth/logout`	refreshToken	success message

Protected API endpoints (require Authorization header)

Method	Path	Auth	Notes
GET	/api/health	Any logged-in user	Quick check, still rate-limited
GET	/api/data	Any logged-in user	Free + pro tier, cached response
GET	/api/premium	pro or admin only	Blocked for free users (403)

Standard headers on every /api/* response

```
X-RateLimit-Limit:     100                  # total allowed per hour
X-RateLimit-Remaining: 73                   # requests left
X-RateLimit-Reset:     1700003600           # unix timestamp, window reset
Retry-After:           847                  # only present on 429 responses
```

Error response format — consistent across the whole API

```
{
  "error": "Human readable message",
  "code":  "MACHINE_READABLE_CODE"    // optional, useful for clients
}

// Examples:
{ "error": "Invalid credentials.", "code": "AUTH_INVALID" }
{ "error": "Rate limit exceeded.", "code": "RATE_LIMIT_EXCEEDED" }
{ "error": "This endpoint requires a pro account.", "code": "FORBIDDEN_ROLE" }
```

■ *Keeping a consistent error shape across your whole API is good practice and a great thing to mention in interviews — it shows you think about API design, not just individual endpoints.*

PART C

Building It Step by Step

7. Step 1 — Project Setup & Dependencies

7.1 Initialize the project

```
mkdir rate-limited-api && cd rate-limited-api
npm init -y

# Core dependencies
npm install express jsonwebtoken bcryptjs pg ioredis dotenv

# Dev dependencies
npm install --save-dev nodemon jest supertest
```

7.2 package.json scripts

```
{
  "name": "rate-limited-api",
  "version": "1.0.0",
  "main": "src/server.js",
  "scripts": {
    "start": "node src/server.js",
    "dev":   "nodemon src/server.js",
    "test":  "jest"
  },
  "dependencies": {
    "express":      "^4.19.0",
    "jsonwebtoken": "^9.0.0",
    "bcryptjs":     "^2.4.3",
    "pg":           "^8.11.0",
    "ioredis":      "^5.4.0",
    "dotenv":       "^16.4.0"
  },
  "devDependencies": {
    "nodemon":  "^3.1.0",
    "jest":     "^29.7.0",
    "supertest":"^6.3.0"
  }
}
```

7.3 .env file

```
# .env
NODE_ENV=development
PORT=3000

ACCESS_TOKEN_SECRET=replace_with_a_long_random_string_min_32_chars
REFRESH_TOKEN_SECRET=replace_with_a_different_long_random_string
ACCESS_TOKEN_EXPIRY=15m
REFRESH_TOKEN_EXPIRY=7d

DATABASE_URL=postgresql://postgres:postgres@localhost:5432/ratelimitdb
```

```
REDIS_HOST=localhost
REDIS_PORT=6379
```

■ *Generate a strong secret quickly with: node -e "console.log(require('crypto').randomBytes(48).toString('hex'))" — run it twice for two different secrets.*

7.4 .gitignore

```
node_modules/
.env
.env.local
*.log
coverage/
.DS_Store
```

8. Step 2 — Database & Redis Connections

8.1 src/config/index.js — centralize all env vars

```
// src/config/index.js
require('dotenv').config()

module.exports = {
    nodeEnv: process.env.NODE_ENV || 'development',
    port:    parseInt(process.env.PORT) || 3000,

    accessTokenSecret:   process.env.ACCESS_TOKEN_SECRET,
    refreshTokenSecret:  process.env.REFRESH_TOKEN_SECRET,
    accessTokenExpiry:   process.env.ACCESS_TOKEN_EXPIRY  || '15m',
    refreshTokenExpiry:  process.env.REFRESH_TOKEN_EXPIRY || '7d',

    databaseUrl: process.env.DATABASE_URL,

    redisHost: process.env.REDIS_HOST || 'localhost',
    redisPort: parseInt(process.env.REDIS_PORT) || 6379,

    rateLimits: {
        free:  100,
        pro:   1000,
        admin: Infinity,
    },
}
```

8.2 src/db/postgres.js

```
// src/db/postgres.js
const { Pool } = require('pg')
const config = require('../config')

const pool = new Pool({
    connectionString: config.databaseUrl,
    max: 10,
    idleTimeoutMillis: 30000,
    connectionTimeoutMillis: 2000,
})

pool.on('error', (err) => {
    console.error('Unexpected PostgreSQL error:', err)
})

module.exports = pool
```

8.3 src/db/redis.js

```
// src/db/redis.js
const Redis = require('ioredis')
```

```
const config = require('../config')

const redis = new Redis({
    host: config.redisHost,
    port: config.redisPort,
    retryStrategy: (times) => Math.min(times * 100, 3000),
})

redis.on('connect', () => console.log('Redis connected'))
redis.on('error',   (err) => console.error('Redis error:', err))

module.exports = redis
```

■ *Both files export a single shared connection (Pool / Redis client). Every other file in the project imports from here — never create a second connection elsewhere.*

9. Step 3 — JWT Utilities

This file is the single source of truth for creating and verifying tokens. Every other file uses these functions instead of calling jsonwebtoken directly.

```
// src/utils/jwt.js
const jwt = require('jsonwebtoken')
const config = require('../config')

function generateAccessToken(payload) {
    // payload = { userId, email, role }
    return jwt.sign(payload, config.accessTokenSecret, {
        expiresIn: config.accessTokenExpiry,
    })
}

function generateRefreshToken(payload) {
    // Only userId — refresh token doesn't need email/role
    return jwt.sign({ userId: payload.userId }, config.refreshTokenSecret, {
        expiresIn: config.refreshTokenExpiry,
    })
}

function verifyAccessToken(token) {
    return jwt.verify(token, config.accessTokenSecret)
    // Throws TokenExpiredError or JsonWebTokenError on failure
}

function verifyRefreshToken(token) {
    return jwt.verify(token, config.refreshTokenSecret)
}

module.exports = {
    generateAccessToken,
    generateRefreshToken,
    verifyAccessToken,
    verifyRefreshToken,
}
```

src/utils/asyncHandler.js — avoid repetitive try/catch

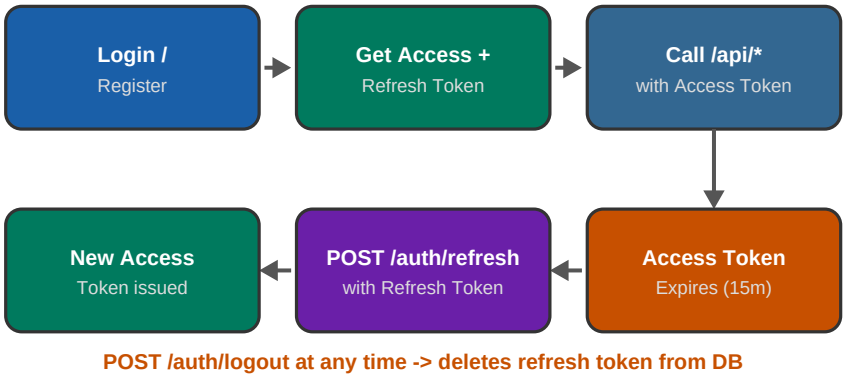
Express does not automatically catch errors thrown inside async route handlers. This wrapper does it for you once, instead of writing try/catch in every single route.

```
// src/utils/asyncHandler.js
function asyncHandler(fn) {
    return (req, res, next) => {
        Promise.resolve(fn(req, res, next)).catch(next)
    }
}
```

```
module.exports = asyncHandler

// Usage in a route:
const asyncHandler = require('../utils/asyncHandler')

router.get('/data', asyncHandler(async (req, res) => {
    const result = await someAsyncDbCall()   // if this throws, asyncHandler catches it
    res.json(result)
}))
```

■ *Without asyncHandler, an error thrown inside an async route handler would crash the process or hang the request. This one small utility prevents an entire category of bugs.*

10. Step 4 — Auth Routes (Register, Login, Refresh, Logout)



The full JWT lifecycle for this project — from login to refresh to logout

10.1 src/controllers/authController.js — register

```
// src/controllers/authController.js
const bcrypt = require('bcryptjs')
const pool   = require('../db/postgres')
const { generateAccessToken, generateRefreshToken } = require('../utils/jwt')

async function register(req, res) {
    const { email, password } = req.body

    if (!email || !password) {
        return res.status(400).json({ error: 'Email and password are required.' })
    }
    if (password.length < 8) {
        return res.status(400).json({ error: 'Password must be at least 8 characters.' })
    }

    const existing = await pool.query('SELECT id FROM users WHERE email = $1', [email])
    if (existing.rows.length > 0) {
        return res.status(409).json({ error: 'Email already registered.' })
    }

    const passwordHash = await bcrypt.hash(password, 12)

    const result = await pool.query(
        `INSERT INTO users (email, password_hash, role)
```

```
         VALUES ($1, $2, 'free') RETURNING id, email, role`,
        [email, passwordHash]
    )
    const user = result.rows[0]

    const accessToken  = generateAccessToken({ userId: user.id, email: user.email, role: user.role })
    const refreshToken = generateRefreshToken({ userId: user.id })

    await pool.query(
        `INSERT INTO refresh_tokens (user_id, token, expires_at)
         VALUES ($1, $2, NOW() + INTERVAL '7 days')`,
        [user.id, refreshToken]
    )

    res.status(201).json({ accessToken, refreshToken, user })
}

module.exports = { register }
```

10.2 login

```
// src/controllers/authController.js (continued)

async function login(req, res) {
    const { email, password } = req.body

    if (!email || !password) {
        return res.status(400).json({ error: 'Email and password are required.' })
    }

    const result = await pool.query(
        'SELECT id, email, password_hash, role FROM users WHERE email = $1',
        [email]
    )
    const user = result.rows[0]

    // Same error message whether user doesn't exist or password is wrong
    // — prevents attackers from learning which emails are registered
    if (!user) {
        return res.status(401).json({ error: 'Invalid credentials.' })
    }

    const match = await bcrypt.compare(password, user.password_hash)
    if (!match) {
        return res.status(401).json({ error: 'Invalid credentials.' })
    }

    const accessToken  = generateAccessToken({ userId: user.id, email: user.email, role: user.role })
    const refreshToken = generateRefreshToken({ userId: user.id })

    await pool.query(
        `INSERT INTO refresh_tokens (user_id, token, expires_at)
         VALUES ($1, $2, NOW() + INTERVAL '7 days')`,
        [user.id, refreshToken]
    )
    await pool.query('UPDATE users SET last_login = NOW() WHERE id = $1', [user.id])

    res.json({ accessToken, refreshToken })
}
```

10.3 refresh

```
async function refresh(req, res) {
    const { refreshToken } = req.body

    if (!refreshToken) {
        return res.status(400).json({ error: 'Refresh token required.' })
    }

    let decoded
    try {
        decoded = verifyRefreshToken(refreshToken)
    } catch (err) {
```

```
        return res.status(401).json({ error: 'Invalid or expired refresh token.' })
    }

    // Confirm it's still valid in the DB (not logged out)
    const tokenResult = await pool.query(
        `SELECT id FROM refresh_tokens
         WHERE token = $1 AND user_id = $2 AND expires_at > NOW()`,
        [refreshToken, decoded.userId]
    )
    if (tokenResult.rows.length === 0) {
        return res.status(401).json({ error: 'Refresh token not found or expired.' })
    }

    const userResult = await pool.query(
        'SELECT id, email, role FROM users WHERE id = $1 AND is_active = true',
        [decoded.userId]
    )
    const user = userResult.rows[0]
    if (!user) {
        return res.status(401).json({ error: 'User not found or inactive.' })
    }

    const newAccessToken = generateAccessToken({
        userId: user.id, email: user.email, role: user.role
    })

    res.json({ accessToken: newAccessToken })
}
```

10.4 logout

```
async function logout(req, res) {
    const { refreshToken } = req.body

    if (!refreshToken) {
        return res.status(400).json({ error: 'Refresh token required.' })
    }

    await pool.query('DELETE FROM refresh_tokens WHERE token = $1', [refreshToken])

    res.json({ message: 'Logged out successfully.' })
}

module.exports = { register, login, refresh, logout }
```

10.5 src/routes/auth.js — wire it up

```
// src/routes/auth.js
const express = require('express')
const asyncHandler = require('../utils/asyncHandler')
const { register, login, refresh, logout } = require('../controllers/authController')

const router = express.Router()

router.post('/register', asyncHandler(register))
router.post('/login',    asyncHandler(login))
router.post('/refresh',  asyncHandler(refresh))
router.post('/logout',   asyncHandler(logout))

module.exports = router
```

11. Step 5 — Auth Middleware

This middleware protects every route under /api. It reads the Authorization header, verifies the JWT, and attaches the decoded user to req.user so later middleware and route handlers can use it.

```
// src/middleware/auth.js
const { verifyAccessToken } = require('../utils/jwt')

function authenticate(req, res, next) {
    const authHeader = req.headers['authorization']

    if (!authHeader) {
        return res.status(401).json({ error: 'Authorization header missing.', code: 'AUTH_MISSING' })
    }

    const parts = authHeader.split(' ')
    if (parts.length !== 2 || parts[0] !== 'Bearer') {
        return res.status(401).json({ error: 'Use format: Bearer <token>', code: 'AUTH_FORMAT' })
    }

    const token = parts[1]

    try {
        const decoded = verifyAccessToken(token)
        req.user = decoded   // { userId, email, role, iat, exp }
        next()
    } catch (err) {
        if (err.name === 'TokenExpiredError') {
            return res.status(401).json({ error: 'Access token expired.', code: 'TOKEN_EXPIRED' })
        }
        return res.status(401).json({ error: 'Invalid token.', code: 'TOKEN_INVALID' })
    }
}

module.exports = authenticate
```

■ *This middleware never touches the database. JWT verification is pure cryptographic math — that is exactly why it scales so well. The role and email are already inside the token from when it was issued.*

12. Step 6 — Role-Based Access Control

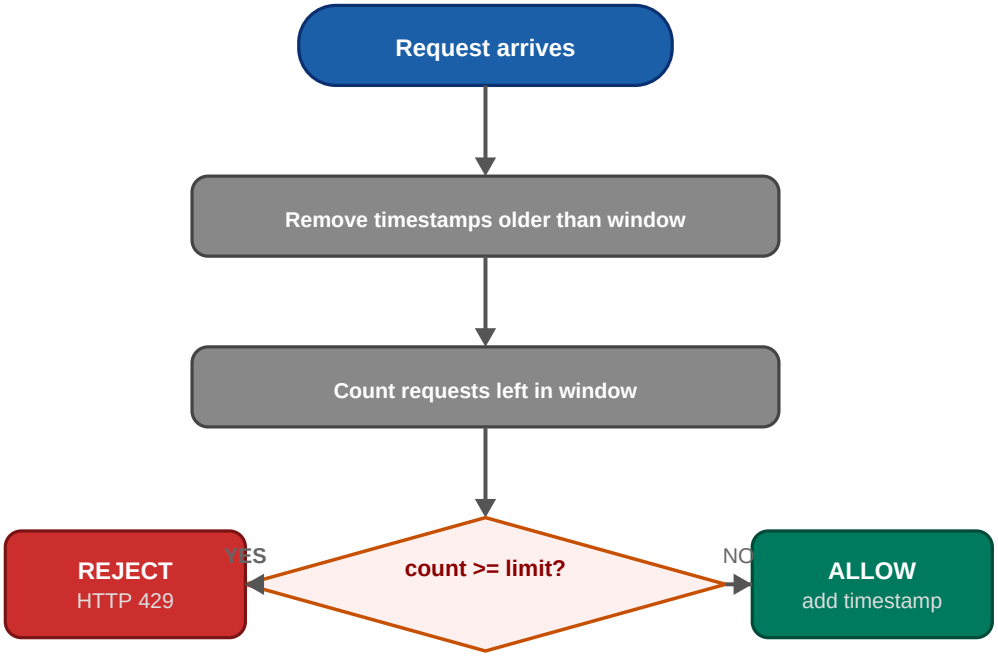
```
// src/middleware/roles.js

function requireRole(...allowedRoles) {
    return (req, res, next) => {
        if (!req.user) {
            return res.status(401).json({ error: 'Not authenticated.' })
        }

        if (!allowedRoles.includes(req.user.role)) {
            return res.status(403).json({
                error: `Requires one of these roles: ${allowedRoles.join(', ')}`,
                code: 'FORBIDDEN_ROLE',
                yourRole: req.user.role,
            })
        }

        next()
    }
}

module.exports = { requireRole }
```

Usage — this runs AFTER authenticate, since it depends on req.user already being set:

```
const authenticate     = require('../middleware/auth')
const { requireRole } = require('../middleware/roles')

router.get('/premium',
    authenticate,
    requireRole('pro', 'admin'),
    asyncHandler(async (req, res) => {
        res.json({ data: 'premium content here' })
    })
)
```

13. Step 7 — Redis Sliding Window Rate Limiter

This is the centerpiece of the project. Refer back to your JWT + Redis guide for the full conceptual explanation — this chapter gives you the final, wired-in implementation.



Decision flow for every request hitting the rate limiter

13.1 src/middleware/rateLimiter.js

```
// src/middleware/rateLimiter.js
const redis  = require('../db/redis')
const config = require('../config')

async function checkRateLimit(userId, role) {
    const limit       = config.rateLimits[role] ?? config.rateLimits.free
    const windowMs    = 60 * 60 * 1000          // 1 hour
    const now         = Date.now()
    const windowStart = now - windowMs
    const key         = `ratelimit:user:${userId}`

    const pipeline = redis.pipeline()
    pipeline.zremrangebyscore(key, 0, windowStart)   // drop old entries
    pipeline.zcard(key)                              // count current entries
    pipeline.zadd(key, now, `${now}-${Math.random()}`) // record this request
    pipeline.expire(key, Math.ceil(windowMs / 1000)) // cleanup safeguard

    const results = await pipeline.exec()
    const countBeforeThisRequest = results[1][1]
```

```
    return {
        allowed:   countBeforeThisRequest < limit,
        limit,
        remaining: Math.max(0, limit - countBeforeThisRequest - 1),
        resetAt:   now + windowMs,
    }
}

function rateLimitMiddleware() {
    return async (req, res, next) => {
        try {
            const { userId, role } = req.user
            const result = await checkRateLimit(userId, role)

            res.set({
                'X-RateLimit-Limit':     result.limit,
                'X-RateLimit-Remaining': result.remaining,
                'X-RateLimit-Reset':     Math.ceil(result.resetAt / 1000),
            })

            if (!result.allowed) {
                const retryAfter = Math.ceil((result.resetAt - Date.now()) / 1000)
                res.set('Retry-After', retryAfter)
                return res.status(429).json({
                    error: 'Rate limit exceeded.',
                    code:  'RATE_LIMIT_EXCEEDED',
                    limit: result.limit,
                    resetAt: new Date(result.resetAt).toISOString(),
                })
            }

            next()
        } catch (err) {
            // Redis down? Fail OPEN — don't block all traffic because of an
            // infra issue. Log it so you know it happened.
            console.error('Rate limiter error:', err)
            next()
        }
    }
}

module.exports = rateLimitMiddleware
```

■ *This is the exact same algorithm from your JWT + Redis PDF, now using req.user.role from the JWT instead of a hardcoded value — that's the missing link between the two pieces.*

14. Step 8 — Response Caching Middleware

This is what the resume bullet 'reducing PostgreSQL load by ~40%' refers to. Cache GET responses in Redis so repeated identical requests skip the database entirely.

```
// src/middleware/cache.js
const redis = require('../db/redis')

function cacheMiddleware(ttlSeconds = 60) {
    return async (req, res, next) => {
        if (req.method !== 'GET') return next()   // only cache GET requests

        const cacheKey = `cache:${req.user.userId}:${req.originalUrl}`

        try {
            const cached = await redis.get(cacheKey)
            if (cached) {
                res.set('X-Cache', 'HIT')
                return res.json(JSON.parse(cached))
            }
        } catch (err) {
            console.error('Cache read error:', err)
            // fall through to DB on cache errors
        }

        // Monkey-patch res.json to cache the response on the way out
        const originalJson = res.json.bind(res)
        res.json = (body) => {
            res.set('X-Cache', 'MISS')
            redis.setex(cacheKey, ttlSeconds, JSON.stringify(body))
                 .catch(err => console.error('Cache write error:', err))
            return originalJson(body)
        }

        next()
    }
}

module.exports = cacheMiddleware
```

Usage on a route:

```
const cacheMiddleware = require('../middleware/cache')

router.get('/data',
    authenticate,
    rateLimitMiddleware(),
    cacheMiddleware(60),     // cache for 60 seconds
    asyncHandler(async (req, res) => {
        const result = await pool.query('SELECT * FROM some_table')
        res.json(result.rows)   // this call gets intercepted and cached
    })
```

)

■ *The cache key includes userId so users never see each other's cached data. TTL of 60 seconds is a reasonable starting point — tune based on how often the underlying data changes.*

15. Step 9 — Async Request Logging

Every request gets logged to PostgreSQL — but the logging itself must never slow down the response. This is done by firing the insert without awaiting it.

```
// src/middleware/requestLogger.js
const pool = require('../db/postgres')

function requestLogger(req, res, next) {
    const startTime = Date.now()

    // 'finish' fires after the response has been sent to the client
    res.on('finish', () => {
        const responseTime = Date.now() - startTime

        // Fire and forget — NOT awaited, so it never delays the response
        pool.query(
            `INSERT INTO request_logs
             (user_id, ip, endpoint, method, status_code, response_time)
             VALUES ($1, $2, $3, $4, $5, $6)`,
            [
                req.user?.userId || null,
                req.ip,
                req.originalUrl,
                req.method,
                res.statusCode,
                responseTime,
            ]
        ).catch(err => console.error('Request logging failed:', err))
    })

    next()
}

module.exports = requestLogger
```

■ *The key design decision here: logging must NEVER be on the critical path of the response. We attach to the 'finish' event and don't await the query — if the DB insert fails or is slow, the client never notices.*

16. Step 10 — Protected API Routes

```
// src/controllers/apiController.js
const pool = require('../db/postgres')

async function health(req, res) {
    res.json({ status: 'ok', user: req.user.email, role: req.user.role })
}

async function getData(req, res) {
    res.json({
        message: `Hello ${req.user.email}`,
        role: req.user.role,
        timestamp: new Date().toISOString(),
    })
}

async function getPremiumData(req, res) {
    res.json({ message: 'This is premium content, only for pro and admin users.' })
}

module.exports = { health, getData, getPremiumData }


// src/routes/api.js
const express = require('express')
const authenticate       = require('../middleware/auth')
const rateLimitMiddleware = require('../middleware/rateLimiter')
const cacheMiddleware     = require('../middleware/cache')
const { requireRole }     = require('../middleware/roles')
const asyncHandler        = require('../utils/asyncHandler')
const { health, getData, getPremiumData } = require('../controllers/apiController')

const router = express.Router()

// Apply auth + rate limiting to EVERY route in this file
router.use(authenticate)
router.use(rateLimitMiddleware())

router.get('/health', asyncHandler(health))

router.get('/data', cacheMiddleware(60), asyncHandler(getData))

router.get('/premium',
    requireRole('pro', 'admin'),
    asyncHandler(getPremiumData)
)

module.exports = router
```

17. Step 11 — Wiring app.js Together

Every piece you built comes together here. This is the file that defines the entire middleware pipeline shown in Chapter 3.

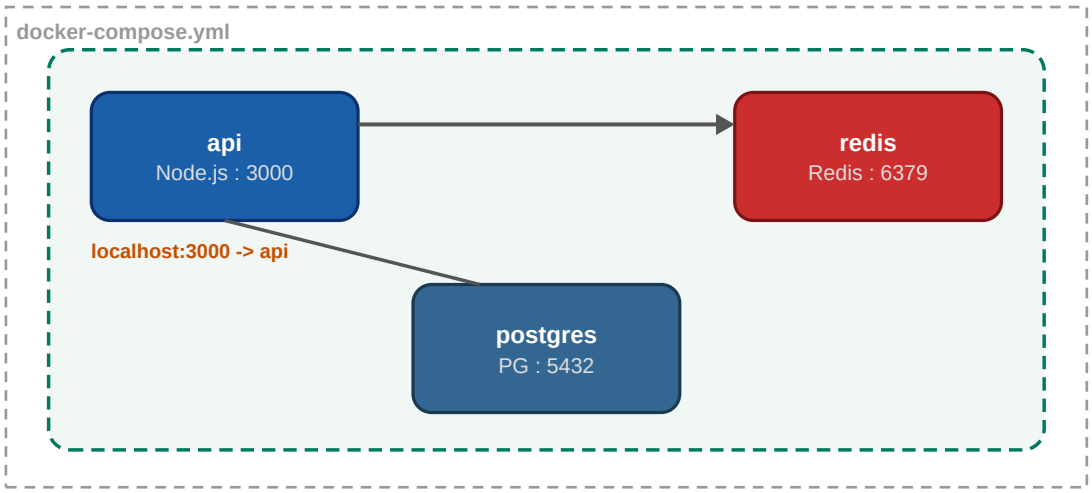
```
// src/app.js
const express        = require('express')
const authRoutes     = require('./routes/auth')
const apiRoutes      = require('./routes/api')
const requestLogger   = require('./middleware/requestLogger')

const app = express()

app.use(express.json())
app.use(requestLogger)   // logs every request, including public auth routes

app.use('/auth', authRoutes)   // public
app.use('/api',  apiRoutes)    // protected (auth applied inside api.js)

// 404 handler
app.use((req, res) => {
    res.status(404).json({ error: 'Route not found.', code: 'NOT_FOUND' })
})

// Global error handler — catches anything asyncHandler passes to next(err)
app.use((err, req, res, next) => {
    console.error(err)
    res.status(500).json({ error: 'Internal server error.', code: 'SERVER_ERROR' })
})

module.exports = app


// src/server.js
const app    = require('./app')
const config = require('./config')

app.listen(config.port, () => {
    console.log(`Server running on port ${config.port} [${config.nodeEnv}]`)
})
```

■ *Splitting app.js (the Express app definition) from server.js (the part that actually listens) is a small but important pattern — it lets you import the app into test files without starting a real server. You'll use this in Chapter 19.*

PART D
Shipping It

18. Dockerizing the Project

Full detail on every one of these concepts is in your Docker guide. This chapter has the final files specific to this project.



The three services this project runs via docker compose up

18.1 Dockerfile

```
FROM node:20-alpine
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --only=production
COPY . .
EXPOSE 3000
CMD ["node", "src/server.js"]
```

18.2 .dockerignore

```
node_modules/
.env
.git/
*.log
tests/
coverage/
```

18.3 docker-compose.yml

```
version: '3.8'

services:
  api:
    build: .
    container_name: rate-limited-api
```

```
    ports:
      - "3000:3000"
    environment:
      - NODE_ENV=production
      - PORT=3000
      - ACCESS_TOKEN_SECRET=${ACCESS_TOKEN_SECRET}
      - REFRESH_TOKEN_SECRET=${REFRESH_TOKEN_SECRET}
      - DATABASE_URL=postgresql://postgres:${POSTGRES_PASSWORD}@postgres:5432/ratelimitdb
      - REDIS_HOST=redis
      - REDIS_PORT=6379
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_healthy
    restart: unless-stopped

  postgres:
    image: postgres:15-alpine
    environment:
      POSTGRES_DB:       ratelimitdb
      POSTGRES_USER:     postgres
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./migrations/init.sql:/docker-entrypoint-initdb.d/init.sql
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      timeout: 5s
      retries: 5

  redis:
    image: redis:alpine
    ports:
      - "6379:6379"
    volumes:
      - redis_data:/data
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 5

volumes:
  postgres_data:
  redis_data:
```

18.4 Start the whole stack

```
docker compose up --build
```

```
curl http://localhost:3000/api/health   # will 401 without a token, that's correct
```

19. Testing Your API — Manual & Automated

19.1 Manual testing with curl — the full flow

```
# 1. Register
curl -X POST http://localhost:3000/auth/register \
  -H "Content-Type: application/json" \
  -d '{"email":"test@example.com","password":"password123"}'
# Save the accessToken and refreshToken from the response

# 2. Call a protected route
curl http://localhost:3000/api/data \
  -H "Authorization: Bearer <accessToken>"

# 3. Try without a token (should 401)
curl http://localhost:3000/api/data

# 4. Try premium as a free user (should 403)
curl http://localhost:3000/api/premium \
  -H "Authorization: Bearer <accessToken>"

# 5. Hammer the rate limit (run 101 times quickly to trigger 429)
for i in $(seq 1 101); do
  curl -s -o /dev/null -w "%{http_code}\n" http://localhost:3000/api/data \
    -H "Authorization: Bearer <accessToken>"
done

# 6. Refresh
curl -X POST http://localhost:3000/auth/refresh \
  -H "Content-Type: application/json" \
  -d '{"refreshToken":"<refreshToken>"}'

# 7. Logout
curl -X POST http://localhost:3000/auth/logout \
  -H "Content-Type: application/json" \
  -d '{"refreshToken":"<refreshToken>"}'
```

19.2 Automated tests with Jest + Supertest

```
// tests/api.test.js
const request = require('supertest')
const app = require('../src/app')

describe('Auth flow', () => {
    let accessToken, refreshToken
    const testEmail = `test${Date.now()}@example.com`

    test('registers a new user', async () => {
        const res = await request(app)
            .post('/auth/register')
            .send({ email: testEmail, password: 'password123' })
```

```
        expect(res.status).toBe(201)
        expect(res.body.accessToken).toBeDefined()
        accessToken  = res.body.accessToken
        refreshToken = res.body.refreshToken
    })

    test('rejects protected route without token', async () => {
        const res = await request(app).get('/api/data')
        expect(res.status).toBe(401)
    })

    test('allows protected route with valid token', async () => {
        const res = await request(app)
            .get('/api/data')
            .set('Authorization', `Bearer ${accessToken}`)
        expect(res.status).toBe(200)
        expect(res.headers['x-ratelimit-limit']).toBeDefined()
    })

    test('blocks free user from premium route', async () => {
        const res = await request(app)
            .get('/api/premium')
            .set('Authorization', `Bearer ${accessToken}`)
        expect(res.status).toBe(403)
    })

    test('refreshes access token', async () => {
        const res = await request(app)
            .post('/auth/refresh')
            .send({ refreshToken })
        expect(res.status).toBe(200)
        expect(res.body.accessToken).toBeDefined()
    })

    test('logs out successfully', async () => {
        const res = await request(app)
            .post('/auth/logout')
            .send({ refreshToken })
        expect(res.status).toBe(200)
    })
})


# Run tests
npm test
```

20. Build Order & Milestones Checklist

Build in this exact order. Each milestone is independently testable before moving to the next — don't write everything then test at the end.

#	Milestone	How to verify it works
1	Project setup, .env, dependencies installed	npm run dev starts without errors
2	PostgreSQL + Redis running (Docker or local)	psql and redis-cli connect successfully
3	users table created, schema runs	SELECT * FROM users; returns empty result, no error
4	POST /auth/register works	curl returns 201 with accessToken
5	POST /auth/login works	curl returns 200 with tokens
6	authenticate middleware blocks unauthenticated calls	GET /api/data without token returns 401
7	authenticate middleware allows valid token	GET /api/data with token returns 200
8	POST /auth/refresh works	Returns new accessToken from refreshToken
9	POST /auth/logout works	Old refreshToken no longer works after logout
10	requireRole blocks free users from /premium	Free user gets 403, pro user gets 200
11	Rate limiter returns headers	Response has X-RateLimit-* headers
12	Rate limiter blocks after limit hit	101st request in an hour returns 429
13	Caching middleware works	Second identical GET shows X-Cache: HIT
14	Request logging works	request_logs table has rows after API calls
15	Full app runs in Docker Compose	docker compose up --build works end to end
16	All Jest tests pass	npm test shows all green

■ *Resist the urge to build all middleware before testing any of it. Test after EVERY milestone. This catches bugs immediately instead of in a tangle of five features at once.*

21. Common Bugs and How to Fix Them

JWT 'invalid signature' error

```
# Cause: ACCESS_TOKEN_SECRET differs between when token was signed and verified
# Often happens after restarting with a different .env, or typo in env var name

# Fix: confirm the same secret is used everywhere
console.log(process.env.ACCESS_TOKEN_SECRET)  # add temporarily to debug
# Make sure .env is loaded BEFORE config/index.js is required anywhere
```

Rate limiter always returns count = 0

```
# Cause: redis.pipeline() results need index [1] not the raw value
# results[i] = [error, value] — you need results[i][1], not results[i]

const requestCount = results[1][1]   // correct
const requestCount = results[1]      // WRONG — this is [error, value]
```

req.user is undefined inside rate limiter

```
# Cause: rateLimitMiddleware() registered BEFORE authenticate
# Fix: authenticate must always run first

router.use(authenticate)             # 1st
router.use(rateLimitMiddleware())    # 2nd — needs req.user.role
```

Refresh token still works after logout

```
# Cause: DELETE query in logout() didn't match correctly,
# or you're checking the wrong table/column

# Debug: check it was actually deleted
SELECT * FROM refresh_tokens WHERE token = '...';
# Should return 0 rows after logout
```

Cached response returned for a different user

```
# Cause: cache key didn't include userId
# Fix: always scope cache keys per user

const cacheKey = `cache:${req.user.userId}:${req.originalUrl}`  # correct
const cacheKey = `cache:${req.originalUrl}`                      # WRONG — shared across users
```

Connection refused to postgres/redis inside Docker

```
# Cause: using localhost instead of service name inside Docker network
DATABASE_URL=postgresql://postgres:5432/db   # WRONG inside docker-compose
DATABASE_URL=postgresql://postgres:5432/db   # service name 'postgres', correct
```

Tests hang and never finish

```
# Cause: pg Pool or Redis connection kept open after tests finish
# Fix: close connections after tests

afterAll(async () => {
    await pool.end()
    await redis.quit()
})
```

22. Interview Questions and Answers

Q: Walk me through what happens when a request hits a protected endpoint.

A: The request first goes through express.json() to parse the body. Then the authenticate middleware reads the Authorization header, verifies the JWT signature, and attaches the decoded payload to req.user — this requires no database call since JWT verification is pure cryptography. Next, the rate limiter middleware checks Redis using a sliding window algorithm based on req.user.role to see if the user has exceeded their hourly limit. If a role check is needed, requireRole verifies req.user.role matches what's allowed. Finally the route handler runs the actual business logic.

Q: Why did you choose a sliding window over a fixed window for rate limiting?

A: A fixed window resets at fixed clock boundaries, like the top of every hour. That creates a burst vulnerability: a user could send 100 requests at 9:59 and another 100 at 10:00, getting 200 requests in two seconds. A sliding window tracks actual request timestamps in a Redis sorted set, so the window continuously slides forward with time — there's no boundary to exploit.

Q: Why use both PostgreSQL and Redis instead of just one database?

A: They serve different needs. PostgreSQL stores durable data — user accounts and refresh tokens — that must survive restarts and support relational queries with foreign keys. Redis stores ephemeral, fast-changing data — rate limit counters and cached responses — where in-memory speed matters more than durability, and losing this data on a restart is acceptable.

Q: How do you prevent your rate limiter from blocking all traffic if Redis goes down?

A: I wrapped the rate limit check in a try/catch. If Redis throws an error, I log it and call next() anyway — failing open rather than failing closed. The trade-off is that during a Redis outage, rate limiting is temporarily disabled rather than the entire API going down. For a production system you'd also want alerting on this failure path.

Q: Why do you log requests asynchronously instead of awaiting the insert?

A: Logging is not part of the business logic the client cares about — it should never slow down or risk failing the actual response. I attach a listener to the response's 'finish' event and fire the database insert without awaiting it. If the insert fails or is slow, the client already has their response and never notices.

Q: What would you change to make this production-ready for real scale?

A: A few things: move secrets to a proper secret manager instead of .env files, add structured logging (e.g. Winston or Pino) instead of console.log, add a connection pool size tuned to expected load, consider read replicas for PostgreSQL if read traffic grows, and move request logs to a time-series-friendly store if log volume gets large since PostgreSQL isn't ideal for very high-volume append-only logs at scale.

Q: How would you test this without manually curling every endpoint?

A: I wrote automated tests with Jest and Supertest that exercise the full auth flow — register, access a protected route, attempt and get blocked from a role-restricted route, refresh the token, and log out. Supertest lets me call the Express app directly without starting a real server, so tests run fast and don't need a live port.

23. Quick Reference Cheatsheet

File responsibility map

File	Responsibility
config/index.js	All environment variables, read once
db/postgres.js	Single shared pg Pool
db/redis.js	Single shared ioredis client
utils/jwt.js	Sign and verify access/refresh tokens
utils/asyncHandler.js	Wrap async routes, forward errors to next()
middleware/auth.js	Verify JWT, attach req.user
middleware/roles.js	Check req.user.role against allowed roles
middleware/rateLimiter.js	Sliding window check via Redis sorted sets
middleware/cache.js	Cache GET responses in Redis per-user
middleware/requestLogger.js	Fire-and-forget log insert on 'finish'
controllers/authController.js	register, login, refresh, logout logic
controllers/apiController.js	Protected route business logic
routes/auth.js	Maps /auth/* paths to authController
routes/api.js	Maps /api/* paths, applies auth + rate limit
app.js	Wires everything, exported for tests
server.js	Actually starts listening on PORT

Middleware order (never change this)

```
express.json() -> requestLogger -> [authenticate -> rateLimiter -> requireRole] -> route handl
                                   (only inside /api routes, auth routes skip the bracketed part)
```

Commands you'll run constantly

Task	Command
Start dev server	`npm run dev`
Run tests	`npm test`

Start full stack	`docker compose up --build`
View API logs	`docker compose logs -f api`
Connect to postgres	`docker compose exec postgres psql -U postgres ratelimitdb`
Connect to redis	`docker compose exec redis redis-cli`
Check a user's rate limit key	`ZCARD ratelimit:user:1  (inside redis-cli)`
Reset everything	`docker compose down -v && docker compose up --build`

You now have the complete blueprint for Project 3 — build it milestone by milestone.
Every line of code here maps directly back to your resume bullet points.

Prepared for Saksham Tiwari · Sir M Visvesvaraya Institute of Technology